

Hierarchical vs Sequential 프로세스 비교

1. 멀티에이전트 프로세스의 개념

멀티에이전트 시스템에서는 여러 Agent를 구성하는 것만으로 충분하지 않다.

여러 Agent를 어떤 순서로 실행하고, 이전 Agent의 결과를 어떻게 다음 Agent에게 전달하며, 문제가 발생했을 때 다시 이전 단계로 돌아갈 것인지 결정해야 한다.

이러한 Agent 실행 및 제어 방식을 멀티에이전트 프로세스라고 할 수 있다.

대표적인 프로세스 방식으로 다음 두 가지가 있다.

- Sequential 프로세스
- Hierarchical 프로세스

두 방식의 가장 중요한 차이는 "다음에 어떤 Agent를 실행할 것인가를 미리 정해 놓는가, 실행 중에 동적으로 결정하는가"이다.

Sequential

Agent A
↓
Agent B
↓
Agent C
↓
종료

Hierarchical

Manager
↙ ↓ ↘
Agent A Agent B Agent C
↑ | |
└───┘
|
Manager
|
다음 결정

Sequential은 고정된 실행 흐름을 사용하고, Hierarchical은 Manager 또는 Supervisor가 현재 상태를 확인한 후 다음 작업을 동적으로 결정한다.

2. Sequential 프로세스

2.1 개념

Sequential 프로세스는 Agent의 실행 순서가 사전에 정해져 있는 구조이다.

예를 들어 콘텐츠 작성 시스템을 다음과 같이 구성할 수 있다.

```
Writer
  ↓
Editor
  ↓
Fact Checker
  ↓
Finalize
```

각 Agent는 자신의 작업을 수행한 후 정해진 다음 Agent에게 결과를 전달한다.

즉, 전체 실행 흐름이 미리 정의되어 있다.

```
A → B → C → D
```

실행 중에 $A \rightarrow B \rightarrow C$ 로 진행하다가 C에서 문제가 발견되더라도 $C \rightarrow A$ 또는 $C \rightarrow B$ 와 같은 역방향 경로가 없다면 처음부터 다시 실행하지 않는다.

3. Sequential 프로세스의 특징

Sequential 프로세스의 핵심 특징은 다음과 같다.

실행 순서가 고정된다

```
Writer
  ↓
Editor
  ↓
Fact Checker
  ↓
Finalize
```

각 Agent의 역할이 명확하다

각 단계가 담당하는 작업이 명확하게 정의된다.

Writer

→ 초안 작성

Editor

→ 문체 및 가독성 개선

Fact Checker

→ 사실관계 검증

Finalize

→ 최종 결과 확정

실행 흐름이 단순하다

프로세스가 직선형이므로 전체 실행 구조를 이해하기 쉽다.

실행 비용을 예측하기 쉽다

각 Agent가 일반적으로 한 번씩 실행되므로 LLM 호출 횟수와 실행 시간이 비교적 예측 가능하다.

4. Sequential 프로세스의 문제점

Sequential 구조에서는 뒤 단계에서 문제가 발견되더라도 앞 단계로 돌아갈 수 없는 경우가 많다.

예를 들어 다음과 같은 흐름을 생각할 수 있다.

Writer

↓

Editor

↓

Fact Checker

↓

문제 발견

Fact Checker가 다음과 같은 문제를 발견했다고 가정한다.

"2023년에 설립되었다."

그런데 실제 참고 자료에는 다음과 같이 되어 있다.

"2021년에 설립되었다."

Fact Checker가 문제를 발견했더라도 Sequential 구조에 다음과 같은 연결이 없다면 수정할 수 없다.

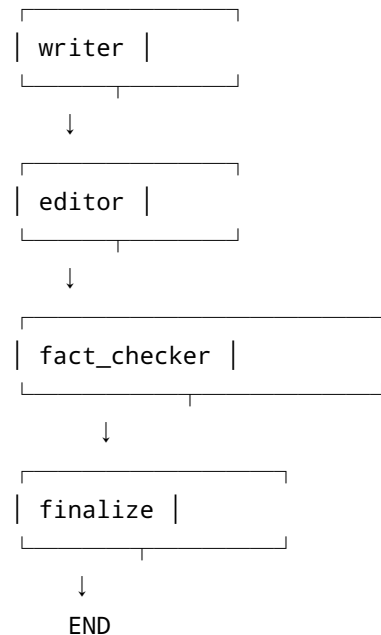
Fact Checker
↓
Writer

따라서 최종 단계에서 문제가 발견되더라도 그대로 종료할 수 있다.

첨부 실습에서도 Sequential 그래프는 `writer → editor → fact_checker → finalize`로 고정되어 있으며, Fact Checker가 문제를 발견하더라도 Writer로 되돌아가는 경로를 구성하지 않는다.

5. Sequential 프로세스 구조

LangGraph 관점에서는 다음과 같은 그래프로 표현할 수 있다.



코드에서는 각 노드 사이의 Edge가 직접 연결된다.

```
seq_graph.add_edge("writer", "editor")
seq_graph.add_edge("editor", "fact_checker")
seq_graph.add_edge("fact_checker", "finalize")
seq_graph.add_edge("finalize", END)
```

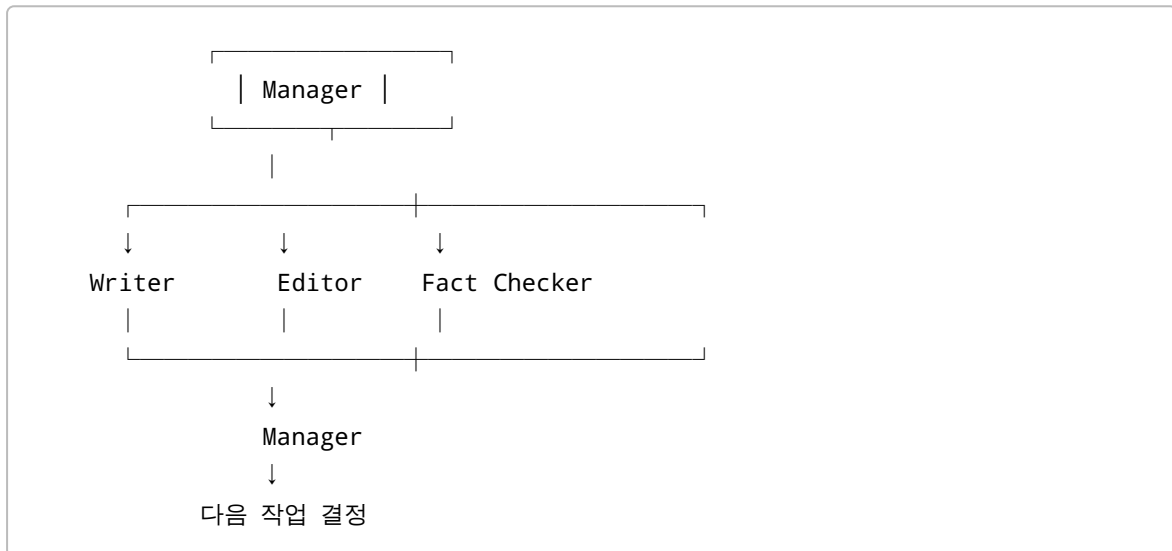
이러한 방식에서는 실행 흐름 자체가 그래프에 고정되어 있다.

6. Hierarchical 프로세스

6.1 개념

Hierarchical 프로세스에서는 Manager 또는 Supervisor 역할을 하는 Agent가 전체 작업을 관리한다.

Manager는 현재 상태를 확인한 후 다음에 실행할 Agent를 결정한다.



따라서 실행 순서가 사전에 완전히 고정되어 있지 않다.

Manager가 현재 결과를 보고 다음 작업을 결정한다.

7. Hierarchical 프로세스의 핵심

Hierarchical 프로세스에서 가장 중요한 요소는 Manager이다.

Manager는 다음과 같은 정보를 확인할 수 있다.

현재 초안 존재 여부
Fact Checker 통과 여부
Editor 승인 여부
남아 있는 피드백
재작업 횟수

그리고 이를 기반으로 다음 작업을 결정한다.

초안이 없다
→ Writer

사실관계 문제가 있다

→ Writer

Fact Checker 검증이 아직 안 됐다

→ Fact Checker

문체 문제가 있다

→ Writer

Editor 검토가 아직 안 됐다

→ Editor

모든 조건을 만족한다

→ Finish

첨부 실습에서는 이러한 판단을 `Decision` 구조로 표현하여 Manager가 `writer`, `editor`, `fact_checker`, `finish` 중 하나를 선택하도록 구성하였다.

8. Hierarchical 프로세스의 실행 예

다음과 같은 요청을 처리한다고 가정한다.

뉴런칩을 소개하는 글을 작성하라.

처음에는 초안이 존재하지 않는다.

Manager:

초안 없음

→ Writer 실행

Writer:

초안 생성

다시 Manager로 돌아간다.

Manager

↓

Fact Checker

Fact Checker가 문제를 발견한다.

문제:
"2023년 설립"이라는 내용은 근거가 없음

다시 Manager로 돌아간다.

Manager
↓
Writer

Writer가 피드백을 반영하여 초안을 다시 작성한다.

다시 Manager가 실행된다.

Manager
↓
Fact Checker

이번에는 사실관계가 통과된다.

Fact Checker
→ OK

Manager가 다시 판단한다.

Manager
↓
Editor

Editor가 문체를 검토한다.

문제가 있다면 다시 Writer로 이동할 수 있다.

Editor
↓
Manager
↓
Writer

문제가 없다면:

Editor
↓

Manager
↓
Finish

이처럼 Hierarchical 구조에서는 실행 과정에서 작업 흐름이 동적으로 변경될 수 있다.

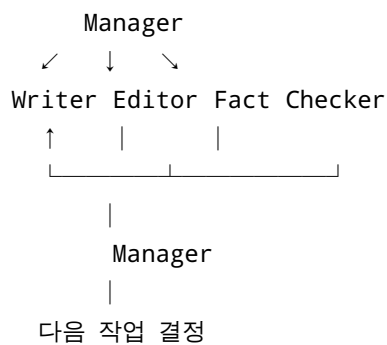
9. Sequential과 Hierarchical의 가장 중요한 차이

두 프로세스의 핵심 차이는 다음과 같다.

Sequential

Writer → Editor → Fact Checker → 종료
고정

Hierarchical



Sequential은 "정해진 순서대로 실행"하는 방식이다.

Hierarchical은 "현재 상태를 확인하고 다음 작업을 결정"하는 방식이다.

10. 피드백 처리 방식 비교

피드백 처리에서 두 방식의 차이가 가장 명확하게 나타난다.

Sequential

Writer
↓
Editor
↓


```

Fact Checker
  ↓
문제 발견
  ↓
Finalize

```

문제를 발견해도 이전 Agent로 돌아가는 경로가 없다면 수정할 수 없다.

Hierarchical

```

Writer
  ↓
Editor
  ↓
Fact Checker
  ↓
문제 발견
  ↓
Manager
  ↓
Writer
  ↓
재작성
  ↓
Fact Checker

```

문제가 해결될 때까지 필요한 Agent를 반복해서 실행할 수 있다.

따라서 Iteration 또는 Feedback Loop가 필요한 업무에서는 Hierarchical 구조가 더 적합할 수 있다.

11. 동일한 Sub-agent를 재사용할 수 있다

Sequential과 Hierarchical을 비교할 때 중요한 점은 Agent 자체를 반드시 다르게 만들 필요가 없다는 것이다.

예를 들어 다음 세 개의 Agent를 공통으로 사용할 수 있다.

```

Writer
Editor
Fact Checker

```

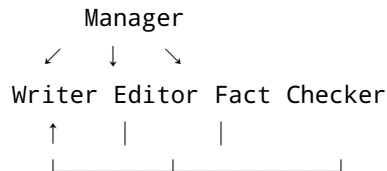
Sequential에서는:

```

Writer
  ↓
Editor
  ↓
Fact Checker

```

Hierarchical에서는:



즉, 차이는 Sub-agent의 구현이 아니라 Sub-agent를 조정하는 프로세스에 있다.

첨부 실습에서도 `writer`, `editor`, `fact_checker` 세 노드를 두 프로세스에서 그대로 재사용하고 프로세스 조정 방식만 변경한다.

12. State의 역할

Hierarchical 구조에서는 현재 시스템의 상태를 Manager가 확인해야 하므로 State 관리가 특히 중요하다.

예를 들어 다음과 같은 State를 사용할 수 있다.

```

class State(TypedDict):
    topic: str
    draft: str
    feedback: List[str]
    editor_approved: bool
    fact_check_ok: bool
    revision_count: int
    next_agent: str
    history: List[str]
    final: str

```

주요 필드는 다음과 같다.

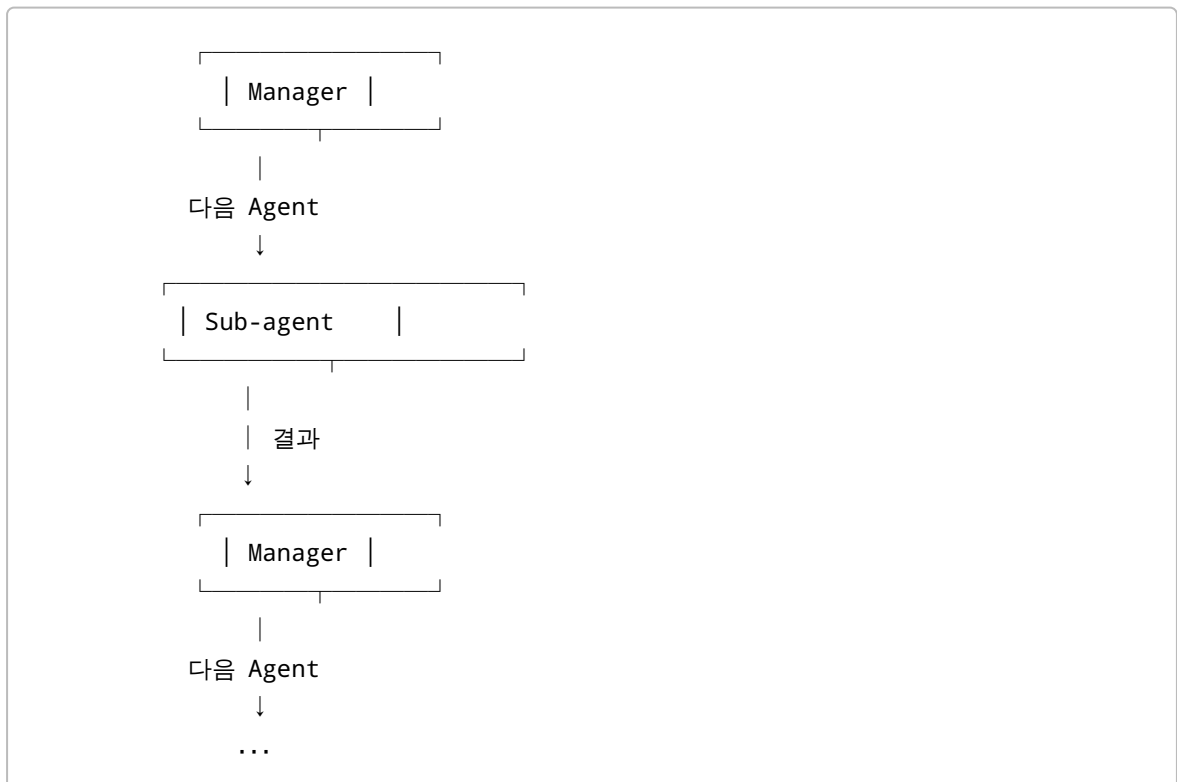
필드	의미
topic	작업 대상
draft	현재 초안

필드	의미
feedback	아직 반영되지 않은 피드백
editor_approved	Editor 승인 여부
fact_check_ok	Fact Checker 통과 여부
revision_count	재작업 횟수
next_agent	Manager가 결정한 다음 Agent
history	실행된 Agent 순서
final	최종 결과

Sequential에서는 상태가 단순히 다음 단계로 전달되는 반면, Hierarchical에서는 Manager가 상태를 분석하여 다음 실행 대상을 결정하는 데 활용한다.

13. Hierarchical의 Loop 구조

Hierarchical 프로세스에서는 Sub-agent가 실행된 후 Manager로 다시 돌아오는 구조가 핵심이다.



따라서 실행 흐름이 다음과 같이 될 수 있다.

```

Manager
→ Writer
→ Manager
  
```

```
→ Fact Checker
→ Manager
→ Writer
→ Manager
→ Editor
→ Manager
→ Finish
```

반드시 모든 Agent를 한 번씩만 실행할 필요가 없다.

14. 무한 루프 방지

Hierarchical 구조에서 반드시 고려해야 하는 문제가 무한 루프이다.

예를 들어 Manager가 계속 Writer를 호출하면 다음과 같은 문제가 발생할 수 있다.

```
Manager
↓
Writer
↓
Manager
↓
Writer
↓
Manager
↓
Writer
↓
...
```

따라서 최대 재작업 횟수를 제한하는 것이 필요하다.

예:

```
MAX_REVISIONS = 4
```

Manager에서 재작업 횟수를 확인한다.

```
if state["revision_count"] >= MAX_REVISIONS:
    return {
        "next_agent": "finish",
```

```

    "final": state["draft"]
}

```

첨부 실습에서도 `revision_count` 를 이용하여 최대 재작업 횟수를 제한하고, 한도를 넘으면 강제로 종료하도록 구현하였다.

15. Sequential과 Hierarchical 비교

비교 항목	Sequential	Hierarchical
실행 순서	고정	동적
제어 방식	사전에 정의된 Graph	Manager가 결정
Agent 반복 실행	일반적으로 없음	가능
이전 단계로 복귀	어려움	가능
Feedback Loop	제한적	지원
구조	단순	복잡
실행 비용	예측하기 쉬움	변동 가능
실행 시간	예측하기 쉬움	상황에 따라 증가
구현 난이도	낮음	높음
디버깅	상대적으로 쉬움	상대적으로 어려움
작업 품질 개선	제한적	반복 검토 가능
적합한 업무	정형화된 Pipeline	복잡하고 동적인 업무

16. Sequential이 적합한 경우

Sequential 프로세스는 작업 순서가 명확하고 변경될 가능성이 낮은 업무에 적합하다.

예를 들어 문서 처리 Pipeline은 다음과 같이 구성할 수 있다.

```

문서 입력
  ↓
Parser
  ↓
Chunker
  ↓
Embedding

```

↓
Vector DB 저장

또는 콘텐츠 생성 시스템에서:

Writer
↓
Editor
↓
Publisher

와 같이 작업 순서가 명확하다면 Sequential 구조가 적합하다.

다음과 같은 특징을 가진 업무에 적합하다.

- 실행 순서가 명확하다.
- 각 단계가 독립적이다.
- 반복 검토가 필요하지 않다.
- 실행 비용과 시간을 예측해야 한다.
- 구현 구조를 단순하게 유지해야 한다.

17. Hierarchical이 적합한 경우

Hierarchical 프로세스는 작업 결과를 평가하고 필요하면 이전 작업을 다시 수행해야 하는 업무에 적합하다.

예를 들어 다음과 같은 업무이다.

초안 작성
↓
검토
↓
문제 발견
↓
수정
↓
재검토
↓
승인

대표적인 사례는 다음과 같다.

콘텐츠 생성

Writer
↓
Reviewer
↓
Writer
↓
Reviewer

코드 생성

Coder
↓
Tester
↓
문제 발견
↓
Coder
↓
Tester

보고서 생성

Research
↓
Writer
↓
Fact Checker
↓
문제 발견
↓
Research / Writer

데이터 분석

Data Analyst
↓
Validator
↓
문제 발견
↓
Data Analyst

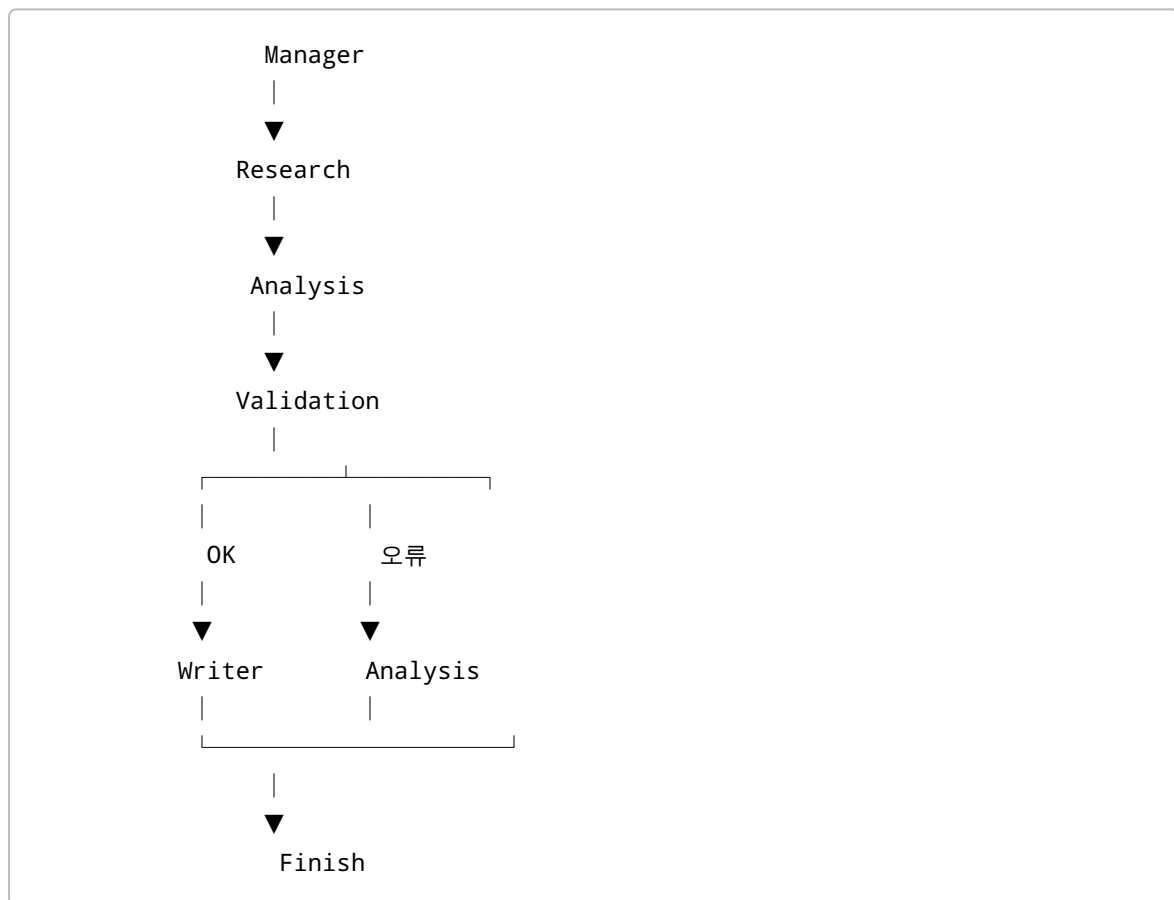
즉, 결과에 대한 검증과 반복 수정이 중요한 업무에서 유용하다.

18. 실무에서는 두 방식을 조합할 수 있다

Sequential과 Hierarchical 중 하나만 사용해야 하는 것은 아니다.

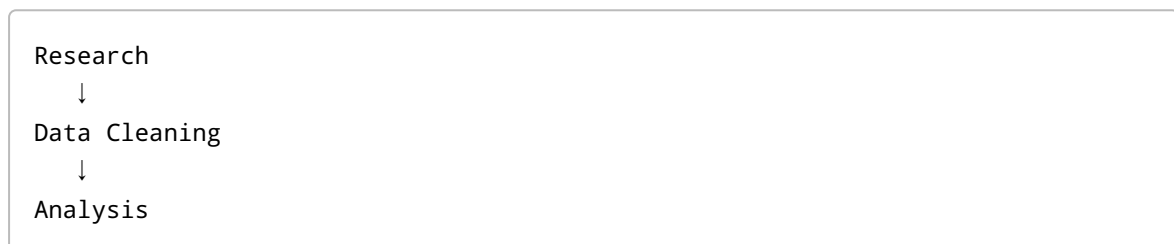
실제 시스템에서는 두 방식을 결합할 수 있다.

예:



여기서 큰 흐름은 Hierarchical 방식으로 관리하지만 일부 내부 작업은 Sequential 방식으로 처리한다.

예를 들어:



라는 Pipeline은 Sequential로 실행하고, 전체 프로젝트의 다음 단계 결정은 Manager가 담당할 수 있다.

이러한 하이브리드 구조는 실제 복잡한 Agent 시스템에서 유용하다.

19. 실행 비용 관점의 비교

Sequential은 실행 횟수가 비교적 일정하다.

예:

```
Writer      1회
Editor      1회
Fact Checker 1회
```

총 3회의 Agent 실행이 발생한다.

반면 Hierarchical은 결과에 따라 실행 횟수가 달라질 수 있다.

정상적인 경우:

```
Manager
→ Writer
→ Manager
→ Fact Checker
→ Manager
→ Editor
→ Manager
→ Finish
```

문제가 발견되면:

```
Manager
→ Writer
→ Manager
→ Fact Checker
→ Manager
→ Writer
→ Manager
→ Fact Checker
→ Manager
→ Editor
→ Manager
→ Finish
```

따라서 Hierarchical은 더 높은 품질을 얻을 수 있는 대신 LLM 호출 횟수와 지연시간이 증가할 수 있다.

20. 설계 시 고려사항

20.1 업무의 정형성

업무 순서가 고정되어 있다면 Sequential이 적합하다.

A → B → C

20.2 피드백 필요성

검증 결과에 따라 수정이 필요하다면 Hierarchical이 적합하다.

A → B → 검증
↓
문제
↓
A 재실행

20.3 비용

반복적인 LLM 호출이 비용에 영향을 주는 경우 Sequential 구조가 유리할 수 있다.

20.4 품질

여러 차례의 검증과 수정이 필요한 경우 Hierarchical 구조가 유리할 수 있다.

20.5 안정적인 종료 조건

Hierarchical에서는 반드시 종료 조건을 정의해야 한다.

모든 검증 통과
또는
최대 재작업 횟수 도달
↓
종료

21. Sequential vs Hierarchical 선택 기준

다음 질문을 통해 프로세스를 선택할 수 있다.

질문 1

작업 순서가 항상 동일한가?

YES → Sequential 고려
NO → Hierarchical 고려

질문 2

검증 결과에 따라 이전 Agent를 다시 실행해야 하는가?

YES → Hierarchical 고려
NO → Sequential 고려

질문 3

실행 비용과 시간을 예측하는 것이 중요한가?

YES → Sequential 고려
NO → Hierarchical 고려

질문 4

품질 향상을 위한 반복적인 검토가 필요한가?

YES → Hierarchical 고려
NO → Sequential 고려

22. 핵심 비교 예제

같은 콘텐츠 제작 문제를 두 방식으로 구현하면 차이가 명확해진다.

Sequential

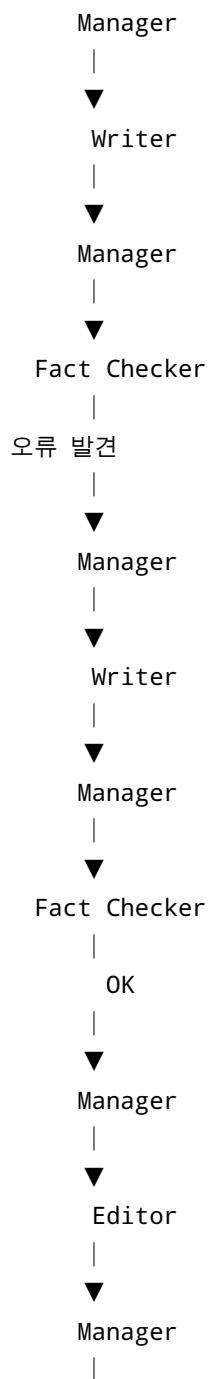
```
Writer
  ↓
Editor
  ↓
Fact Checker
  ↓
Finalize
```

Fact Checker가 오류를 발견해도:

Fact Checker
↓
Finalize

로 종료한다.

Hierarchical



▼
Finish

동일한 Writer, Editor, Fact Checker를 사용하더라도 프로세스 제어 방식에 따라 실행 결과와 실행 횟수가 달라질 수 있다.

23. LangGraph 관점의 핵심 차이

Sequential에서는 일반적인 Edge를 사용하여 실행 순서를 고정한다.

```
graph.add_edge("writer", "editor")
graph.add_edge("editor", "fact_checker")
graph.add_edge("fact_checker", "finalize")
```

따라서:

```
Writer → Editor → Fact Checker → Finalize
```

가 고정된다.

반면 Hierarchical에서는 Manager의 판단 결과에 따라 Conditional Edge를 사용한다.

```
graph.add_conditional_edges(
    "manager",
    route_from_manager,
    {
        "writer": "writer",
        "editor": "editor",
        "fact_checker": "fact_checker",
        "finish": END
    }
)
```

그리고 각 Sub-agent가 다시 Manager로 돌아간다.

```
graph.add_edge("writer", "manager")
graph.add_edge("editor", "manager")
graph.add_edge("fact_checker", "manager")
```

따라서 다음과 같은 동적 흐름이 만들어진다.

```
graph TD; Manager --> Sub-agent; Sub-agent --> Manager; Manager --> Sub-agent; Sub-agent --> Manager; Manager --> Finish;
```

24. 핵심 정리

Sequential과 Hierarchical의 차이는 Sub-agent 자체보다 Agent를 어떻게 조율하는가에 있다.

Sequential은 다음과 같이 동작한다.

```
graph TD; A[정해진 순서] --> B[Agent A]; B --> C[Agent B]; C --> D[Agent C]; D --> E[종료];
```

따라서 구조가 단순하고 실행 비용과 시간이 예측하기 쉽다.

Hierarchical은 다음과 같이 동작한다.

```
graph TD; Manager --> A[현재 상태 분석]; A --> B[다음 Agent 선택]; B --> C[Agent 실행]; C --> D[결과 확인]; D --> Manager;
```

↓
다음 Agent 선택

따라서 실행 흐름이 유연하고 문제가 발견되면 필요한 Agent를 다시 실행할 수 있다.

두 프로세스의 선택 기준을 간단하게 정리하면 다음과 같다.

상황	적합한 프로세스
작업 순서가 명확함	Sequential
단순 Pipeline	Sequential
실행 횟수 예측이 중요함	Sequential
비용을 최소화해야 함	Sequential
반복적인 검토가 필요함	Hierarchical
오류 발생 시 재작업 필요	Hierarchical
Agent 선택이 동적임	Hierarchical
복잡한 의사결정 필요	Hierarchical
품질 검증과 수정이 중요함	Hierarchical

결국 Sequential은 "정해진 절차를 안정적으로 실행하는 구조"이고, Hierarchical은 "Manager가 상황을 판단하면서 필요한 Agent를 동적으로 실행하는 구조"이다.

멀티에이전트 시스템을 설계할 때는 모든 문제를 Hierarchical 구조로 만드는 것이 아니라, 업무의 복잡도와 피드백 필요성에 따라 적절한 프로세스를 선택해야 한다.